

Title: Design and Implementation of Distributed Application using RPC

Aim

To design and implement a **distributed application using Remote Procedure Call (RPC)** where a **client invokes a procedure on a remote server** and receives the result transparently.

Software & Hardware Requirements

Software Requirements

- Linux OS (RHEL / Rocky / Alma / Ubuntu)
- GCC Compiler
- RPC utilities (rpcgen, rpcbind)
- libtirpc library

Hardware Requirements

- Two Linux systems / VMs
 - One as **Server**
 - One as **Client**
 - Network connectivity between client and server
-

Theory

Remote Procedure Call (RPC) is a communication mechanism used in distributed systems that allows a program to execute a procedure on a remote machine as if it were a local function call. RPC hides the complexity of message passing, network communication, and data serialization.

In this experiment:

- The **server** provides a remote procedure to calculate student marks.
- The **client** invokes this procedure remotely.
- The interface is defined using **IDL (.x file)**.
- Stub code is generated using rpcgen.

Algorithm

Server Side

1. Define RPC interface using .x file
2. Generate stub code using rpcgen
3. Implement server-side logic
4. Register service with rpcbind
5. Wait for client requests

Client Side

1. Use generated client stubs
2. Connect to server using IP address
3. Invoke remote procedure
4. Display result returned by server

Program Files

File 1: student.x (RPC Interface Definition);

File2: student_server.c

File 3: student_client.c

Compilation & Execution Steps

◆ Server Side Steps

1. Generate RPC files:

```
rpcgen student.x
```

This command will generate files like:

student.h

student_clnt.c

student_svc.c

student_xdr.c

2. Compile server:

```
gcc -I/usr/include/tirpc -o student_server student_server.c student_svc.c student_xdr.c -ltirpc
```

Start RPC service:

```
sudo systemctl start rpcbind
```

4. Run server:

```
sudo ./student_server
```

◆ Client Side Steps

1. Copy required files from server using scp:

```
scp user@192.168.150.134:/root/rpc-lab* .
```

Note: If SCP Don't Work in your VM then create file using vim editor.

At client side file are student_client.c student_clnt.c student_xdr.c, student.h

2. Compile client:

```
gcc -I/usr/include/tirpc -o student_client student_client.c student_clnt.c student_xdr.c -ltirpc
```

3. Run client:

```
./student_client 192.168.150.134 (Your ServerIP)
```

Output

Student Marks = 50

Conclusion

RPC simplifies distributed programming by allowing transparent execution of procedures on remote systems, making distributed applications easier to develop and maintain.

FAQS

1. What is RPC?
 2. What is rpcgen?
 3. What is the role of rpcbind?
 4. Difference between RPC and RMI
 5. What is a stub?
 6. What is IDL?
-